

# GESTISAC

## Arquitetura do sistema, base de dados, liga??es e fluxos

PDF gerado a partir da auditoria t?cnica e do documento  
docs/35-arquitetura-sistema-base-dados-fluxos.md.

Estado verificado: 2026-06-04. Sem passwords, tokens ou connection strings completas.

---

### Como usar este PDF

Usa este ficheiro para perceber a l?gica real do sistema: quem comunica com quem, quais s?o as tabelas, que liga??es existem e que fluxos de dados suportam os casos de uso principais.

Importante: ? uma fotografia fiel do estado auditado, n?o uma garantia autom?tica se houver altera??es futuras.

## Nota de validade

Este PDF representa o estado auditado e documentado do GESTISAC em 2026-06-04, cruzando o schema real do Supabase/Postgres, o código da API/Web e os testes efetuados em produção. Deve ser lido como uma fotografia técnica fiel do estado verificado nesse momento. Se forem feitas novas migrations, alterações de código, variações Vercel ou mudanças manuais na base de dados, este documento deve ser revisto.

## Arquitetura do sistema, base de dados e fluxos

Estado verificado em 2026-06-04.

Este documento explica como o GESTISAC está ligado entre frontend, API, Vercel, Supabase/Postgres, Codex e dados internos. Não contém passwords, tokens nem connection strings completas.

## Resumo executivo

O GESTISAC está configurado como uma aplicação online gerida:

- ? Frontend Web: Vercel, projeto `gestisac-web`.
- ? Backend API: Vercel, projeto `gestisac-api`.
- ? Base de dados: Supabase PostgreSQL, schema `public`.
- ? Browser do utilizador: fala apenas com a API.
- ? API Rust: fala com Postgres através de `GESTISAC_DATABASE_URL`.
- ? Codex/Supabase MCP: usado apenas para desenvolvimento, auditoria e manutenção.
- ? PC local: usado apenas para desenvolvimento, não para manter produção online.

## Diagrama Mermaid

```
graph LR
    USER["Utilizador no browser"] --> WEB["Vercel Web: gestisac-web"]
    WEB -->|"HTTPS + Bearer token"| API["Vercel API: gestisac-api"]
    API -->|"Postgres connection string"| DB["Supabase PostgreSQL"]
    API -->|"metadata/storage_key"| DOCS["Documentos e ficheiros"]
    GIT["GitHub repo"] --> WEB
    GIT --> API
    CODEX["Codex"] -->|"MCP Supabase"| DB
    VERCEL["Vercel env vars"] --> WEB
    VERCEL --> API
```

## Configuração de runtime

### Frontend Web

O frontend está em `apps/web`.

Variável principal:

### Bloco técnico

`VITE_API_BASE_URL=https://gestisac-api.vercel.app`

Lógica:

- ? O bundle Web chama a API por `VITE_API_BASE_URL`.

- ? Em producao oficial, se a variavel falhar no build, existe fallback seguro apenas para `gestisac-web.vercel.app`.
- ? Clones noutros dominios nao devem usar automaticamente a API principal.
- ? O frontend nunca recebe `GESTISAC_DATABASE_URL`.
- ? O frontend nunca fala diretamente com Supabase/Postgres.

## Backend API

O backend esta em `apps/api`.

Variaveis principais:

## Bloco t?cnico

```
GESTISAC_DATABASE_URL
GESTISAC_ENV
JWT_SECRET
GESTISAC_CORS_ORIGINS
GESTISAC_RUN_MIGRATIONS
GESTISAC_SYNC_ON_STARTUP
GESTISAC_ALLOW_DEMO_SEED
GESTISAC_DATABASE_POOL_MAX
GESTISAC_DOCUMENT_STORAGE_PATH
```

Logica:

- ? `GESTISAC_DATABASE_URL` e obrigatorio.
- ? Em producao, a base de dados ativa e Postgres.
- ? O health check mostra a URL redigida, nunca a password.
- ? A API usa pool Postgres pequeno para ambiente serverless.
- ? SQLx tem cache de prepared statements desligada para compatibilidade com Supabase pooler.
- ? Producao usa Supabase pooler na porta 5432.
- ? `GESTISAC_RUN_MIGRATIONS=false` no fluxo normal de producao.
- ? `GESTISAC_SYNC_ON_STARTUP=false` no fluxo normal de producao.
- ? `GESTISAC_ALLOW_DEMO_SEED=false` em producao.

## Como os componentes comunicam

### Diagrama Mermaid

```
sequenceDiagram
    participant U as Utilizador
    participant W as Web Vercel
    participant A as API Rust Vercel
    participant P as Supabase Postgres

    U->>W: Abre /hq/login
    W->>A: POST /api/auth/login
    A->>P: Procura user e valida password_hash
    P-->>A: User + tenant + active_condominium
    A->>P: Grava session/token hash
    A-->>W: token + refresh_token + PublicUser
    W-->>U: Dashboard autenticado
    W->>A: GET /api/dashboard
    A->>P: Le dados por tenant
    P-->>A: Dados relacionais/snapshots
    A-->>W: JSON seguro
```

## Modelo mental da base de dados

A base de dados tem dois estilos em simultaneo:

- ? Tabelas relacionais: modelo principal, com colunas fortes, foreign keys e soft delete.

? Tabelas \*\_snapshots: payload JSON historico/compatibilidade/arranque para partes ainda nao totalmente normalizadas.

Regras gerais:

? Quase todas as tabelas tem tenant\_id para separar clientes/organizacoes.

? Soft delete usa deleted\_at; apagar no produto normalmente marca como apagado/arquivado.

? metadata jsonb guarda campos flexiveis de dominio.

? payload jsonb nos snapshots guarda uma copia completa do objeto.

? RLS esta ativo nas tabelas publicas.

? Apesar de RLS estar ativo, o browser nao usa Supabase Data API diretamente.

? A autorizacao real do utilizador acontece na API Rust.

? Antes de expor Supabase diretamente no frontend, e preciso criar policies RLS por tenant/user.

## Diagrama ER simplificado

### Diagrama Mermaid

```
erDiagram
    tenants ||--o{ users : "tenant_id"
    tenants ||--o{ condominiums : "tenant_id"
    tenants ||--o{ suppliers : "tenant_id"
    users ||--o{ sessions : "user_id"
    users ||--o{ audit_log : "user_id"
    users ||--o{ condominiums : "manager_user_id"

    condominiums ||--o{ buildings : "condominium_id"
    condominiums ||--o{ fractions : "condominium_id"
    condominiums ||--o{ residents : "condominium_id"
    condominiums ||--o{ condominium_zones : "condominium_id"
    condominium_zones ||--o{ equipment : "zone_id"
    buildings ||--o{ fractions : "building_id"
    fractions ||--o{ residents : "fraction_id"

    condominiums ||--o{ tickets : "condominium_id"
    fractions ||--o{ tickets : "fraction_id"
    residents ||--o{ tickets : "resident_id"
    suppliers ||--o{ tickets : "supplier_id"
    equipment ||--o{ tickets : "equipment_id"
    users ||--o{ tickets : "assigned_worker_id"
    tickets ||--o{ ticket_comments : "ticket_id"
    tickets ||--o{ ticket_attachments : "ticket_id"
    tickets ||--o{ ticket_events : "ticket_id"

    condominiums ||--o{ quotas : "condominium_id"
    fractions ||--o{ quotas : "fraction_id"
    residents ||--o{ quotas : "resident_id"
    quotas ||--o{ payments : "quota_id"
    debts ||--o{ payments : "debt_id"
    payments ||--o{ receipts : "payment_id"
    expenses ||--o{ cash_movements : "expense_id"
    payments ||--o{ cash_movements : "payment_id"
    bank_transactions ||--o{ bank_reconciliations : "bank_transaction_id"
```

## Inventario de tabelas

Todas as 60 tabelas abaixo existem em public.

### Sistema e migracoes

| Tabela           | Funcao                                   | Ligacoes principais |
|------------------|------------------------------------------|---------------------|
| _sqlx_migrations | Historico das migrations SQLx aplicadas. | Sem FK de dominio.  |

## Identidade, tenants, sessoes e auditoria

| Tabela              | Funcao                                                       | Ligacoes principais                                                |
|---------------------|--------------------------------------------------------------|--------------------------------------------------------------------|
| tenants             | Organizacoes/clientes dentro do sistema.                     | Raiz logica de quase todo o modelo.                                |
| tenant_snapshots    | Snapshot JSON de tenants.                                    | Sem FK fisica.                                                     |
| users               | Utilizadores da aplicacao, password hash e condominio ativo. | tenant_id -> tenants.id.                                           |
| user_snapshots      | Snapshot publico/sanitizado do utilizador.                   | Sem FK fisica.                                                     |
| roles               | Roles configuraveis por tenant.                              | tenant_id -> tenants.id.                                           |
| user_roles          | Relacao muitos-para-muitos entre users e roles.              | user_id -> users.id, role_id -> roles.id, tenant_id -> tenants.id. |
| sessions            | Sessoes autenticadas atuais/novas.                           | tenant_id -> tenants.id, user_id -> users.id.                      |
| app_sessions        | Sessoes legadas/compatibilidade.                             | Sem FK fisica.                                                     |
| audit_log           | Eventos auditaveis relacionais.                              | tenant_id -> tenants.id, user_id -> users.id.                      |
| audit_log_snapshots | Snapshot JSON do historico de auditoria.                     | Sem FK fisica.                                                     |

## Condominios e estrutura fisica

| Tabela                | Funcao                                        | Ligacoes principais                                                                          |
|-----------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------|
| condominiums          | Entidade central: condominio/edificio gerido. | tenant_id -> tenants.id, manager_user_id -> users.id.                                        |
| condominium_snapshots | Snapshot JSON de condominios.                 | Sem FK fisica.                                                                               |
| buildings             | Blocos/edificios dentro de um condominio.     | tenant_id -> tenants.id, condominium_id -> condominiums.id.                                  |
| fractions             | Fracoes/unidades.                             | tenant_id -> tenants.id, condominium_id -> condominiums.id, building_id -> buildings.id.     |
| residents             | Moradores/proprietarios associados a fracoes. | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id.     |
| condominium_zones     | Zonas, pisos, areas comuns e locais com QR.   | tenant_id -> tenants.id, condominium_id -> condominiums.id.                                  |
| equipment             | Equipamentos associados a condominio/zona.    | tenant_id -> tenants.id, condominium_id -> condominiums.id, zone_id -> condominium_zones.id. |

## Tickets, avarias e ocorrencias

| Tabela                          | Funcao                                                                      | Ligacoes principais                                                                                                                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tickets                         | Modelo relacional principal para tickets, pedidos, avarias e fluxo tecnico. | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id, resident_id -> residents.id, supplier_id -> suppliers.id, zone_id -> condominium_zones.id, equipment_id -> equipment.id, assigned_worker_id -> users.id, created_by -> users.id, updated_by -> users.id. |
| ticket_comments                 | Comentarios internos/publicos de tickets.                                   | tenant_id -> tenants.id, ticket_id -> tickets.id, author_id -> users.id.                                                                                                                                                                                                                          |
| ticket_attachments              | Anexos de tickets.                                                          | tenant_id -> tenants.id, ticket_id -> tickets.id, uploaded_by -> users.id.                                                                                                                                                                                                                        |
| ticket_events                   | Timeline/eventos de tickets.                                                | tenant_id -> tenants.id, ticket_id -> tickets.id, actor_id -> users.id.                                                                                                                                                                                                                           |
| ticket_snapshots                | Snapshot JSON de tickets.                                                   | Sem FK fisica.                                                                                                                                                                                                                                                                                    |
| ocorrencia_snapshots            | Modelo antigo/compatibilidade de ocorrencias.                               | Sem FK fisica.                                                                                                                                                                                                                                                                                    |
| ocorrencia_comment_snapshots    | Comentarios do modelo antigo de ocorrencias.                                | Sem FK fisica.                                                                                                                                                                                                                                                                                    |
| ocorrencia_attachment_snapshots | Anexos do modelo antigo de ocorrencias.                                     | Sem FK fisica.                                                                                                                                                                                                                                                                                    |

## Operacao, manutencao, vistorias e calendario

| Tabela                | Funcao                              | Ligacoes principais                                                                                                                       |
|-----------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| maintenance_items     | Manutencoes programadas/corretivas. | tenant_id -> tenants.id, condominium_id -> condominiums.id, equipment_id -> equipment.id, created_by -> users.id, updated_by -> users.id. |
| maintenance_snapshots | Snapshot JSON de manutencoes.       | Sem FK fisica.                                                                                                                            |

| Tabela                   | Funcao                                | Ligacoes principais                                                                                                                         |
|--------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| inspections              | Vistorias/inspecoes.                  | tenant_id -> tenants.id, condominium_id -> condominiums.id, assigned_worker_id -> users.id, created_by -> users.id, updated_by -> users.id. |
| inspection_snapshots     | Snapshot JSON de vistorias.           | Sem FK fisica.                                                                                                                              |
| calendar_events          | Eventos de calendario operacional.    | tenant_id -> tenants.id, condominium_id -> condominiums.id, created_by -> users.id, updated_by -> users.id.                                 |
| calendar_event_snapshots | Snapshot JSON de eventos.             | Sem FK fisica.                                                                                                                              |
| assembly_snapshots       | Assembleias ainda em modelo snapshot. | Sem FK fisica.                                                                                                                              |

## Documentos, relatorios e fornecedores

| Tabela             | Funcao                                  | Ligacoes principais                                                                                      |
|--------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------|
| documents          | Metadados de documentos.                | tenant_id -> tenants.id, created_by -> users.id, updated_by -> users.id.                                 |
| document_links     | Liga documentos a entidades do sistema. | tenant_id -> tenants.id, document_id -> documents.id, target_type e target_id fazem ligacao polimorfica. |
| document_snapshots | Snapshot JSON de documentos.            | Sem FK fisica.                                                                                           |
| report_snapshots   | Relatorios ainda em modelo snapshot.    | Sem FK fisica.                                                                                           |
| suppliers          | Fornecedores/prestadores.               | tenant_id -> tenants.id.                                                                                 |
| supplier_snapshots | Snapshot JSON de fornecedores.          | Sem FK fisica.                                                                                           |

## Contabilidade e tesouraria

| Tabela                         | Funcao                                                      | Ligacoes principais                                                                                                                                                                                               |
|--------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| quotas                         | Quotas a cobrar.                                            | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id, resident_id -> residents.id, created_by -> users.id, updated_by -> users.id.                                             |
| quota_snapshots                | Snapshot JSON de quotas.                                    | Sem FK fisica.                                                                                                                                                                                                    |
| payments                       | Pagamentos recebidos.                                       | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id, resident_id -> residents.id, quota_id -> quotas.id, debt_id -> debts.id, created_by -> users.id, updated_by -> users.id. |
| accounting_payment_snapshots   | Snapshot JSON de pagamentos contabilisticos.                | Sem FK fisica.                                                                                                                                                                                                    |
| debts                          | Dividas em aberto.                                          | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id, resident_id -> residents.id, quota_id -> quotas.id, created_by -> users.id, updated_by -> users.id.                      |
| debt_snapshots                 | Snapshot JSON de dividas.                                   | Sem FK fisica.                                                                                                                                                                                                    |
| receipts                       | Recibos emitidos.                                           | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id, resident_id -> residents.id, payment_id -> payments.id, created_by -> users.id, updated_by -> users.id.                  |
| receipt_snapshots              | Snapshot JSON de recibos.                                   | Sem FK fisica.                                                                                                                                                                                                    |
| expenses                       | Despesas/contas a pagar.                                    | tenant_id -> tenants.id, condominium_id -> condominiums.id, supplier_id -> suppliers.id, document_id -> documents.id, created_by -> users.id, updated_by -> users.id.                                             |
| expense_snapshots              | Snapshot JSON de despesas.                                  | Sem FK fisica.                                                                                                                                                                                                    |
| payment_agreements             | Acordos de pagamento de dividas.                            | tenant_id -> tenants.id, condominium_id -> condominiums.id, fraction_id -> fractions.id, resident_id -> residents.id, debt_id -> debts.id, created_by -> users.id, updated_by -> users.id.                        |
| payment_agreement_installments | Prestacoes de um acordo.                                    | tenant_id -> tenants.id, agreement_id -> payment_agreements.id, payment_id -> payments.id.                                                                                                                        |
| payment_agreement_snapshots    | Snapshot JSON de acordos de pagamento.                      | Sem FK fisica.                                                                                                                                                                                                    |
| cash_movements                 | Movimentos de caixa/banco derivados de pagamentos/despesas. | tenant_id -> tenants.id, condominium_id -> condominiums.id, payment_id -> payments.id, expense_id -> expenses.id, created_by -> users.id, updated_by -> users.id.                                                 |
| cash_movement_snapshots        | Snapshot JSON de movimentos de caixa.                       | Sem FK fisica.                                                                                                                                                                                                    |
| bank_transactions              | Movimentos bancarios importados.                            | tenant_id -> tenants.id, condominium_id -> condominiums.id, created_by -> users.id, updated_by -> users.id.                                                                                                       |

| Tabela                        | Funcao                                             | Ligacoes principais                                                                              |
|-------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------|
| bank_transaction_snapshots    | Snapshot JSON de movimentos bancarios.             | Sem FK fisica.                                                                                   |
| bank_reconciliations          | Reconciliacao entre banco e objeto contabilistico. | tenant_id -> tenants.id, bank_transaction_id -> bank_transactions.id, reconciled_by -> users.id. |
| bank_reconciliation_snapshots | Snapshot JSON de reconciliacoes.                   | Sem FK fisica.                                                                                   |
| reserve_fund_snapshots        | Fundo de reserva ainda em modelo snapshot.         | Sem FK fisica.                                                                                   |

## Chat

| Tabela                 | Funcao                                          | Ligacoes principais |
|------------------------|-------------------------------------------------|---------------------|
| chat_message_snapshots | Mensagens de chat guardadas como snapshot JSON. | Sem FK fisica.      |

## Fluxos de dados principais

### Login e sessao

#### Diagrama Mermaid

```

flowchart TD
    A["POST /api/auth/login"] --> B["Normaliza email"]
    B --> C["Procura utilizador em users"]
    C --> D["Valida password com password_hash"]
    D --> E["Gera access token e refresh token"]
    E --> F["Guarda hash da sessao em sessions/app_sessions"]
    F --> G["Devolve PublicUser + tokens ao frontend"]
    G --> H["Frontend guarda sessao para chamadas futuras"]

```

Notas:

- ? A password nunca e devolvida ao frontend.
- ? A API usa password\_hash em users.
- ? user\_snapshots.payload foi sanitizado e nao deve conter passwordHash.

### Dashboard

#### Diagrama Mermaid

```

flowchart TD
    A["Web /hq/dashboard"] --> B["GET /api/dashboard"]
    B --> C["API valida Bearer token"]
    C --> D["Carrega AppStore/hidrata dados Postgres"]
    D --> E["Calcula metricas: condominios, tickets, contabilidade, relatorios"]
    E --> F["Resposta JSON ao frontend"]
    F --> G["Cards e modulos no dashboard"]

```

### Criar/editar condominio

#### Diagrama Mermaid

```

flowchart TD
    A["Formulario Web"] --> B["POST/PUT /api/condominiums"]
    B --> C["API valida autenticacao e permissoes"]
    C --> D["Grava/atualiza condominiums"]
    D --> E["Atualiza condominium_snapshots"]
    E --> F["Regista audit_log/audit_log_snapshots quando aplicavel"]
    F --> G["Frontend atualiza lista/detalhe"]

```

## Ticket/ocorrencia

### Diagrama Mermaid

```
stateDiagram-v2
    [*] --> nova
    nova --> triagem
    triagem --> em_progresso
    em_progresso --> resolvida_por_trabalhador
    resolvida_por_trabalhador --> validacao_hq
    validacao_hq --> fechada
    validacao_hq --> em_progresso
    nova --> cancelada
    fechada --> [*]
    cancelada --> [*]
```

Fluxo:

- ? Um ticket pertence opcionalmente a condominio, fracao, residente, fornecedor, zona ou equipamento.
- ? Pode ter trabalhador atribuido por `assigned_worker_id`.
- ? Comentarios, anexos e eventos ficam em tabelas filhas.
- ? O modelo antigo de ocorrencias ainda existe em `ocorrencia_*_snapshots`.

## Contabilidade

### Diagrama Mermaid

```
flowchart LR
    C["Condominio"] --> Q["Quotas"]
    F["Fracao"] --> Q
    R["Residente"] --> Q
    Q --> P["Payments"]
    Q --> D["Debts"]
    D --> P
    P --> REC["Receipts"]
    P --> CM["Cash movements"]
    E["Expenses"] --> CM
    BT["Bank transactions"] --> BR["Bank reconciliations"]
    P --> BR
    E --> BR
```

Notas:

- ? quotas, payments, debts, receipts, expenses, payment\_agreements, cash\_movements, bank\_transactions e bank\_reconciliations sao o nucleo financeiro relacional.
- ? Alguns objetos financeiros continuam tambem espelhados em snapshots.
- ? `reserve_fund_snapshots` existe, mas nao existe tabela relacional `reserve_funds` no schema atual.

## Documentos

### Diagrama Mermaid

```
flowchart TD
    A["Upload/metadados no frontend"] --> B["API documentos"]
    B --> C["documents"]
    C --> D["document_links"]
    D --> E["Entidade alvo: condominio, ticket, despesa, etc."]
    C --> F["storage_key"]
```

Nota importante:

- ? `documents` guarda metadados e `storage_key`.
- ? Vercel nao deve ser tratado como storage persistente.
- ? Para producao madura, ficheiros binarios devem ir para storage gerido como Supabase Storage ou S3.



## Casos de uso principais

| Caso de uso          | Ator                   | Entrada                                         | Tabelas principais                                            | Resultado                            |
|----------------------|------------------------|-------------------------------------------------|---------------------------------------------------------------|--------------------------------------|
| Entrar no HQ         | Administrador          | Email/password                                  | users, sessions, app_sessions                                 | Token e dashboard autenticado.       |
| Ver dashboard        | Administrador          | Token                                           | condominiums, tickets, snapshots operacionais/financeiros     | Metricas e avisos.                   |
| Gerir condominios    | Administrador          | Dados de condominio                             | condominiums, condominium_snapshots, audit_log                | Lista/detalhe atualizado.            |
| Gerir estrutura      | Administrador          | Blocos, fracoes, residentes, zonas, equipamento | buildings, fractions, residents, condominium_zones, equipment | Estrutura operacional do condominio. |
| Criar ticket         | HQ/cliente/trabalhador | Pedido/avaria                                   | tickets, ticket_events, ticket_snapshots                      | Ticket criado e rastreavel.          |
| Comentar ticket      | HQ/trabalhador/cliente | Comentario                                      | ticket_comments, ticket_events                                | Historico atualizado.                |
| Anexar ficheiro      | HQ/trabalhador         | Ficheiro/metadados                              | ticket_attachments ou documents                               | Anexo associado.                     |
| Gerir contabilidade  | Administrador          | Quotas, pagamentos, recibos, despesas           | quotas, payments, debts, receipts, expenses, cash_movements   | Estado financeiro atualizado.        |
| Reconciliar banco    | Administrador          | Movimento bancario                              | bank_transactions, bank_reconciliations                       | Movimento conciliado.                |
| Consultar documentos | Administrador          | Filtros/entidade                                | documents, document_links                                     | Lista de documentos.                 |
| Auditar atividade    | Administrador/sistema  | Acao de negocio                                 | audit_log, audit_log_snapshots                                | Rasto de operacao.                   |

## Rotas API por modulo

| Modulo              | Rotas principais                                                                                                                                                                                                                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Health/version      | /api/health, /api/version                                                                                                                                                                                                                                                                                              |
| Auth                | /api/auth/login, /api/auth/refresh, /api/auth/logout, /api/me, /api/permissions                                                                                                                                                                                                                                        |
| Dashboard           | /api/dashboard                                                                                                                                                                                                                                                                                                         |
| Chat                | /api/chat/messages                                                                                                                                                                                                                                                                                                     |
| Condominios         | /api/condominiums, /api/condominiums/{id}, subrotas de estrutura, documentos, media, notas e alertas                                                                                                                                                                                                                   |
| Ocorrencias/tickets | /api/tickets, /api/ocorrencias e subrotas de comentarios/transicoes quando aplicavel                                                                                                                                                                                                                                   |
| Contabilidade       | /api/accounting/summary, /api/accounting/overview, /api/accounting/quotas, /api/accounting/payments, /api/accounting/debts, /api/accounting/receipts, /api/accounting/expenses, /api/accounting/payment-agreements, /api/accounting/cash-movements, /api/accounting/bank-transactions, /api/accounting/reconciliations |
| Recursos            | /api/buildings, /api/fractions, /api/residents, /api/suppliers, /api/documents, /api/reports, /api/maintenance, /api/inspections, /api/calendar-events, /api/assemblies                                                                                                                                                |

## Como a API carrega e persiste dados

### Diagrama Mermaid

flowchart TD

```

A["Arranque da API"] --> B["Ler env"]
B --> C["Conectar Postgres"]
C --> D{"GESTISAC_RUN_MIGRATIONS?"}
D -->|"true"| E["Executar SQLx migrations"]
D -->|"false"| F["Nao migrar no arranque"]

```

```

E --> G["Hidratar AppStore"]
F --> G
G --> H["Carregar identidade, condominios, estrutura, operacao, documentos, financeiro"]
H --> I{"GESTISAC_SYNC_ON_STARTUP?"}
I --> J["true"] | J["Reescrever snapshots a partir da store"]
I --> K["false"] | K["Apenas servir API"]

```

Pontos importantes:

- ? A API mantém uma AppStore em memória por instância serverless.
- ? Em produção, a fonte de verdade é Postgres.
- ? Se dados forem alterados diretamente na base, pode ser preciso redeploy/restart para limpar memória quente de uma função serverless antiga.
- ? Endpoints críticos devem ser testados do ponto de vista do utilizador, não apenas com `/api/health`.

## Segurança e limites atuais

O que está correto:

- ? Passwords não vão para o frontend.
- ? Hash de password fica em `users.password_hash`.
- ? `users.metadata` e `user_snapshots.payload` estão sanitizados.
- ? O browser não tem connection string Postgres.
- ? CORS limita origens autorizadas.
- ? RLS está ligado nas tabelas públicas.

O que ainda é risco/roadmap:

- ? RLS está ligado, mas várias tabelas ainda não têm políticas desenhadas para acesso direto.
- ? Supabase Data API não deve ser exposta diretamente ao frontend antes de políticas por tenant/user.
- ? Existem várias tabelas `*_snapshots`; a longo prazo, alguns módulos devem migrar para relacional completo.
- ? Storage binário persistente deve ser formalizado fora do filesystem efêmero da Vercel.
- ? Build Web tem chunk grande a otimizar no futuro.

## Regras para clones futuros

Para criar clone/site novo:

1. Criar novo projeto Supabase ou novo tenant conforme objetivo.
2. Nunca reutilizar secrets da produção principal.
3. Configurar `GESTISAC_DATABASE_URL` no projeto API correspondente.
4. Configurar `VITE_API_BASE_URL` no projeto Web correspondente.
5. Confirmar `GESTISAC_CORS_ORIGINS` com o domínio Web do clone.
6. Executar migrations/schema de forma controlada.
7. Popular dados iniciais sem passwords em metadata JSON.
8. Fazer deploy Web prebuilt pequeno.
9. Fazer deploy API pequeno.
10. Testar login e módulos reais no browser.

## Checklist de verificação

### Bloco técnico

```

curl https://gestisac-api.vercel.app/api/health
GESTISAC_SMOKE_PASSWORD=<password> npm run check:prod-api

```

```
npx vercel logs https://gestisac-api.vercel.app --since 15m --status-code 500 --json --cwd apps/api
pnpm run typecheck:web
pnpm run build:web
pnpm run check:api
node scripts/run-cargo.mjs fmt --manifest-path apps/api/Cargo.toml -- --check
pnpm run clippy:api
pnpm run test:api
```

#### Resultado esperado:

- ? /api/health devolve activeBackend: postgresql.
- ? Login real funciona no browser.
- ? Dashboard, Condomínios, Tickets, Contabilidade, Documentos e Relatórios carregam sem erros.
- ? Sem localhost ou 127.0.0.1 como API de produção.
- ? Sem 500 nos endpoints usados pela UI.
- ? Sem secrets em docs, Git ou bundle frontend.